

Chapter 2: Predicting Ad Clicks on a Social Media Platform

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The explanations, trade-off analysis and diagrams are my own. Notes marked **Beyond the book** are material I added.

The design in one paragraph. For each eligible ad, predict a **calibrated** click probability $\hat{p} = P(\text{click} \mid \text{user, ad, context})$. The auction ranks ads by expected value, $\text{bid} \times \hat{p}$, so the probability itself, not just its ordering, drives revenue. Start with logistic regression on crossed features. Move to a learned-interaction model (DCN or DeepFM) when it measurably beats LR. Use two-tower retrieval only when the ad inventory is too large to score everything. Keep frequency caps and hide/block rules in the serving policy, not in the model. Validate with point-in-time data, a shadow run, a canary and an A/B test.

Ad-click prediction design map

The chapter moves from product behavior and labels to ranking, calibration, and safe rollout.



Minimum-scope principle

Start with a calibrated Logistic Regression baseline. Add interaction modeling or retrieval complexity only when data and scale justify it.

Figure 1: Ad-click prediction design map

1. Scope and product policy

- **Input:** a user, a slot in their feed, and the set of **eligible** ads (targeting, budget and policy checks already passed).
- **Model output:** $\hat{p}$ for each eligible ad.
- **Service action:** choose which ad fills the slot.

Why calibration matters more here than in feed ranking: in a cost-per-click auction the platform's expected revenue from showing an ad is

$$\text{eCPM} = \text{bid}_{\text{CPC}} \times \hat{p} \times 1000.$$

If $\hat{p}$ is $2\times$ too high for one advertiser's ads, they win auctions they shouldn't, and the pricing is wrong. **Ranking quality (AUC) isn't enough: the numbers have to be right.**

Repeated exposure is a policy decision, not a model decision. The model estimates the click probability. The serving policy decides how often the same ad may reappear: no cap, a fixed impression cap (e.g. 3 per user per day), or a time-decay fatigue rule. Hide and block actions

Repeated exposure is a product decision

The model predicts click probability, but serving policy decides how often the same ad may reappear.

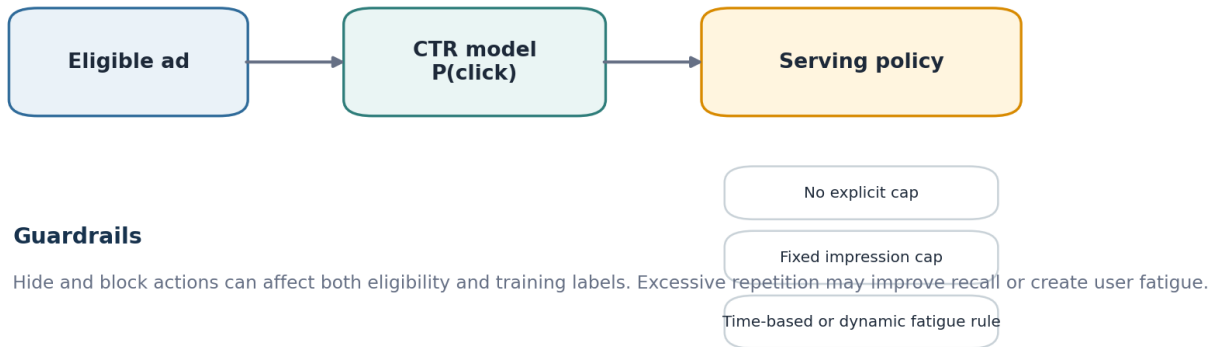


Figure 2: Repeated exposure policy

remove the ad (or advertiser) for that user and also become training signals. Include “times this user has seen this ad” as a model feature so the model can learn fatigue, but enforce hard caps in policy.

2. Data and labels

Family	Examples
ad / campaign	ad ID, advertiser ID, category, creative text/image embeddings, campaign age
user / context	user ID, demographics, language, device, location, time of day
interaction history	user's CTR on this category or advertiser, recent clicks, conversions, dwell
impression	position, times already shown, surface

One training example = one impression, with features as they were at impression time.

- **Positive:** a click within a window (e.g. 30 min) after the impression.
- **Negative:** an impression with no click in the window. This is the transparent default.
- **The ambiguity:** “viewed but not clicked” isn’t the same as “not interested”. The user may have scrolled past without seeing the ad, or clicked a day later. Options:
 - require a minimum on-screen time before counting a negative;
 - down-weight uncertain negatives;
 - treat non-clicks as *unlabeled* and use positive-unlabeled (PU) learning.

Start transparent and add these only if hidden positives measurably hurt calibration or ranking.

- **Delayed feedback:** clicks and especially conversions arrive late. Training on too-recent data mislabels future positives as negatives. Wait for the window, or model the delay explicitly.

Data and label construction

Clicks are clear positives. Viewed-but-unclicked impressions are ambiguous.

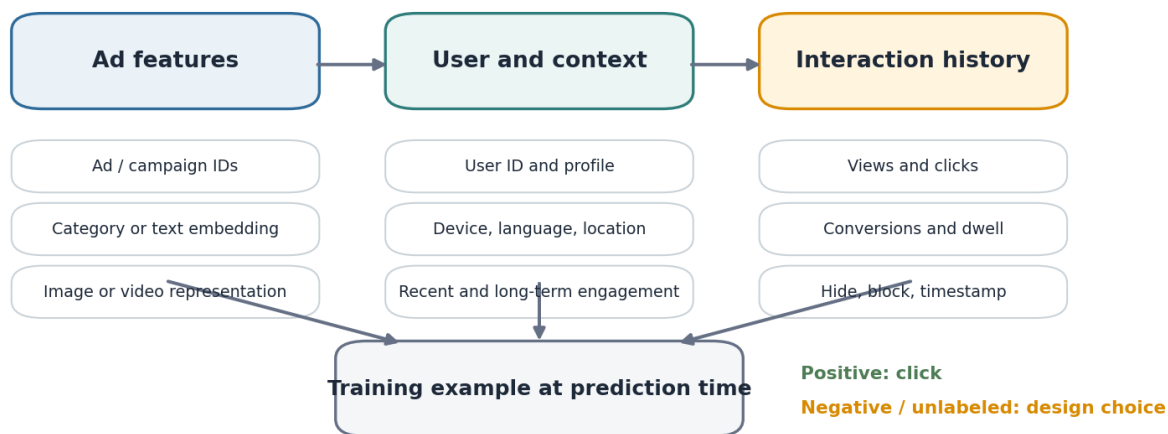


Figure 3: Data and labels

3. Metrics

Question	Metric
are the probabilities accurate?	log loss (binary cross-entropy); calibration ratio $\sum \hat{p} / \sum y$ (should be ≈ 1)
better than a named baseline?	normalized entropy
does the score separate clicks from non-clicks?	ROC-AUC; PR-AUC because clicks are rare
does the business improve?	CTR, conversions, revenue per 1,000 impressions
is the user experience protected?	hide/block rate, ad load, session length (guardrails)

Normalized entropy (He et al., 2014) divides the model's log loss by the log loss of always predicting the average CTR $\bar{p}$:

$$\text{NE} = \frac{-\frac{1}{N} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]}{-[\bar{p} \log \bar{p} + (1 - \bar{p}) \log(1 - \bar{p})]}.$$

Lower is better, and $\text{NE} < 1$ means the model beats the constant predictor. Example: with $\bar{p} = 2\%$ the denominator is 0.098 nats, so a model with log loss 0.090 has $\text{NE} = 0.918$. Always say which baseline you normalize by: the constant CTR, the production model, or the LR baseline.

Note. > Beyond the book: correcting for negative downsampling

Clicks are $\sim 1\text{-}2\%$ of impressions, so it's common to keep only a fraction w of negatives (e.g. $w = 0.1$). The model then learns inflated odds, by a factor $1/w$. Recover the true probability with

Metrics answer different questions

No single offline metric represents ranking quality, calibration, revenue, and user experience.

Probability quality	Binary cross-entropy	Are predicted probabilities numerically useful?
Relative probability quality	Normalized cross-entropy	Did the model beat a clearly named baseline?
Discrimination	ROC-AUC / PR-AUC	Can the score separate clicks from non-clicks?
Online product	CTR / conversion / revenue lift	Did the system improve business outcomes?
User guardrail	Hide or block rate	Did monetization gains harm user experience?

Normalized cross-entropy is interpretable only when the comparison baseline is explicit: constant historical CTR, current production model, or Logistic Regression.

Figure 4: Metrics

$$q = \frac{\hat{p}}{\hat{p} + (1 - \hat{p})/w}.$$

For example, $\hat{p} = 0.10$ with $w = 0.1$ becomes $q \approx 0.011$. Skipping this step breaks the eCPM auction even when AUC is unchanged.

4. Model alternatives

Model	How it handles feature interactions	Strength	Cost
logistic regression	none unless you add them by hand	fast, interpretable, calibrated by construction, easy online updates	misses interactions
LR + feature crosses	hand-made crosses (user_country × ad_category), hashed into buckets	captures the crosses you know about	combinatorial feature explosion; only crosses you thought of
GBDT	tree splits learn nonlinear interactions	strong on dense features	awkward with sparse IDs; slow to update incrementally
GBDT + LR	tree leaves become one-hot features for LR (He et al., 2014)	automatic cross discovery + LR's easy updates	two-stage pipeline
factorization machine	every pair i, j gets weight $\langle \mathbf{v}_i, \mathbf{v}_j \rangle$ from low-rank embeddings	pairwise interactions even for rare pairs, $O(kn)$ time	only second-order

Model	How it handles feature interactions	Strength	Cost
DeepFM	FM component + a deep MLP, sharing embeddings	low- and high-order interactions, no manual crosses	more latency and tuning
DCN	cross layers $\mathbf{x}_{l+1} = \mathbf{x}_0 \mathbf{x}_l^\top \mathbf{w}_l + \mathbf{b}_l + \mathbf{x}_l$ build explicit crosses of degree $l + 1$, plus a deep path	explicit, bounded-degree crosses at low cost	more complex than LR
xDeepFM	compressed interaction network: explicit vector-wise crosses	richer explicit interactions	heavier still

Factorization machine:

$$\hat{y} = w_0 + \sum_i w_i x_i + \sum_{i < j} \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j.$$

The pairwise term can be rewritten as $\frac{1}{2} \sum_f [(\sum_i v_{if} x_i)^2 - \sum_i v_{if}^2 x_i^2]$, which costs $O(kn)$ instead of $O(kn^2)$. Because each feature has its own embedding, the model can score a pair (user_country, ad_category) it never saw together in training.

My plan: LR (with a few known crosses) as the anchor and the control arm. DCN and DeepFM as the challengers. Adopt one if it improves NE by a meaningful margin (e.g. $\geq 0.5\%$ relative, which is significant at ad scale), stays calibrated after correction, and fits the latency budget.

5. Where the two-tower model belongs

A two-tower model scores with a dot product between a user embedding and an ad embedding. That makes it great for **retrieval** (ad embeddings precomputed, ANN search over millions of ads) and weak as the **final CTR model**: there are no cross-features, and its scores aren't calibrated probabilities. So:

- **small inventory** (thousands of eligible ads per request): skip retrieval and score them all with the CTR model;
- **large inventory**: two-tower / ANN retrieval $\rightarrow$ a few hundred candidates $\rightarrow$ the rich, calibrated CTR model.

Candidate sources (ANN, collaborative filtering, eligibility/campaign rules, others) are merged and deduplicated, and each ad keeps a feature for which sources found it.

6. Keeping the model fresh

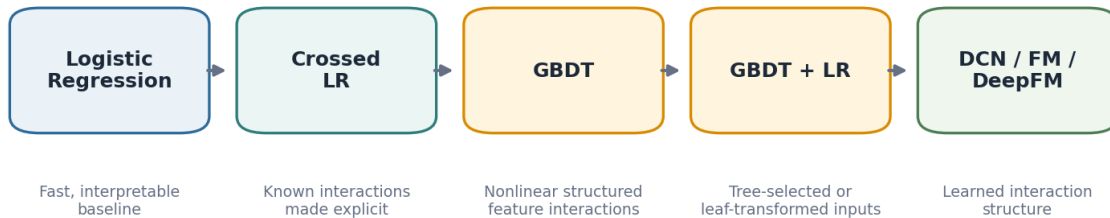
Ads churn fast: new campaigns every day, and user interests and seasonality shift. Stale models lose calibration first. Triggers for an update: calibration ratio drifting from 1, campaign turnover, feature distribution shift, falling online metrics. Update options, from lightest to heaviest:

1. **frequent retraining / warm-start fine-tuning** on the latest window (often daily or more);
2. **regularization** toward the previous weights, to limit forgetting;
3. **replay** a sample of older data;
4. **adapters / new capacity** for new ad types.

Use the lightest option that fixes the measured drift.

A practical model progression

The options represent increasing interaction capacity, not a mandatory sequence.



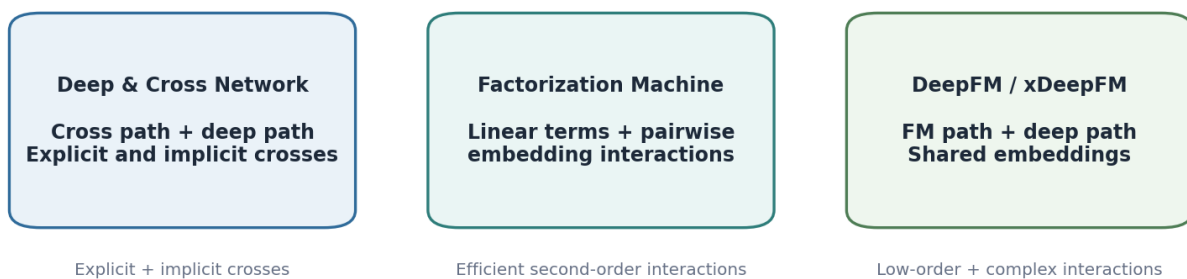
Starting experiment

Use Logistic Regression as the anchor, then compare DCN and DeepFM when automatic feature interaction learning is valuable.

Figure 5: Model progression

How richer models learn interactions

Each family encodes user-ad relationships differently.



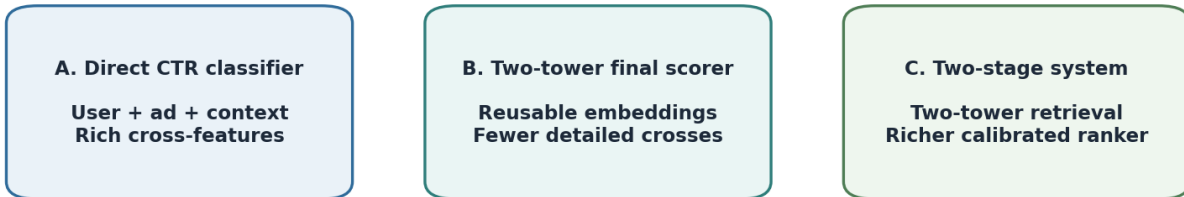
Selection question

Which interaction patterns matter, how much interpretability is needed, and what serving latency is acceptable?

Figure 6: Interaction models

Where should a two-tower model sit?

Retrieval and final CTR ranking may need different representations.



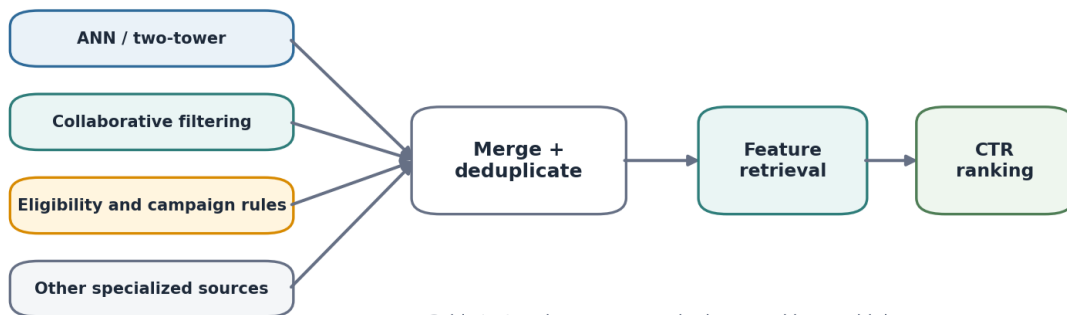
Starting recommendation

Use the two-tower model for scalable candidate retrieval and preserve a richer classifier for calibrated final ranking.

Figure 7: Two-tower placement

Candidate generation and final ranking

Candidate sources may be combined, then deduplicated and enriched before CTR scoring.



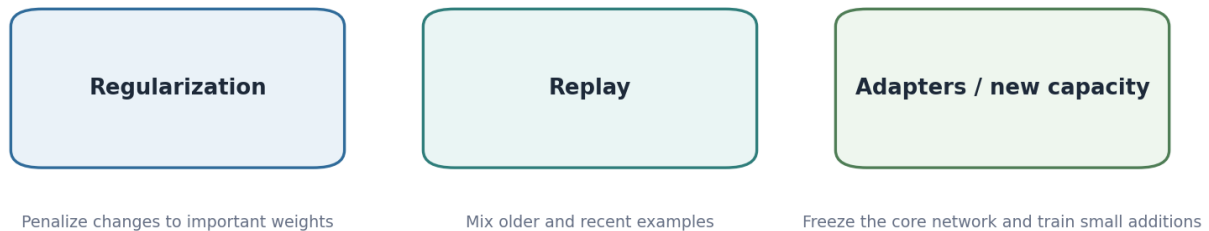
Cold-start and coverage can be improved by combining sources.

Preserve source indicators as ranking features when multiple sources retrieve the same ad.

Figure 8: Candidate generation

Continual learning is a drift response

Choose the smallest update strategy that addresses observed drift without unnecessary forgetting.



Trigger examples

CTR calibration drift, campaign turnover, changing user interests, feature distribution shift, or declining online metrics.

They can be used alone or combined; pick the smallest one that fixes the measured problem.

Figure 9: Continual learning

7. Validation, rollout and calibration

- **Shadow:** the new model scores live requests without affecting auctions. Compare score distributions, calibration and latency with production.
- **Canary:** it serves a small slice. Check for errors, spend anomalies and advertiser complaints.
- **A/B test:** measures causal impact on CTR, revenue and hide rate. Watch **budget effects:** treatment and control share advertiser budgets, which can bias results, so budget-split or advertiser-level randomization may be needed.
- **Calibration methods**, all fitted on a recent holdout:

Method	Fits	Use when
Platt scaling	$\sigma(a \cdot \text{logit} + b)$	miscalibration is a smooth shift or scale
isotonic regression	any monotone map	lots of data, irregular miscalibration
temperature scaling	a single T dividing logits	neural models that are over-confident

8. Leakage

Every feature and label is computed relative to the **impression time**:

- features use only events before the impression (e.g. “user’s clicks in the last 7 days *before* this impression”);
- labels use an explicit window after it;
- train/validation/test are split **chronologically**, e.g. train on days 1-28, validate on day 29, test on day 30.

A random split lets the model see tomorrow’s behavior of the same campaign, and the offline metrics come out better than what you’ll ever see in production.

9. Reference architecture

Area	Starting choice	Switch when (measured)
retrieval	none (score all eligible ads)	eligible ads per request exceed what the CTR model can score in budget
calibration	Platt on a daily holdout	the reliability curve is non-sigmoid → isotonic
freshness	daily warm-start retraining	NE decays measurably within a day → more frequent or online updates
leakage	point-in-time features, chronological splits	never optional

11. Questions and answers

Q: Why optimize log loss rather than AUC?

AUC ignores calibration: multiplying every prediction by 2 leaves AUC unchanged and breaks the auction. Log loss (and NE) penalizes both poor ranking and wrong probabilities.

Q: How do I handle a brand-new ad with no history?

Use content features (creative embeddings, category, advertiser history) so the model can generalize, add a small exploration budget (e.g. Thompson sampling on an uncertainty estimate) so new ads get impressions, and back off to advertiser- or category-level CTR priors.

Q: Which normalized-entropy baseline should I report?

State it explicitly. The constant average CTR is standard (NE < 1 means “better than knowing nothing about the user”). For launch decisions, compare with the current production model.

Q: Should frequency be a feature or a rule?

Both. The feature lets the model learn that the 5th showing is less likely to be clicked. The rule enforces a hard limit that the product guarantees users, whatever the model says.

Q: What does the A/B test need beyond CTR?

Revenue and advertiser value (conversions, cost per acquisition), user guardrails (hide/block rate, session time), and a check that treatment and control didn’t compete for the same budgets.

Validation, rollout, and probability calibration

Shadow, canary, and A/B testing provide different evidence.

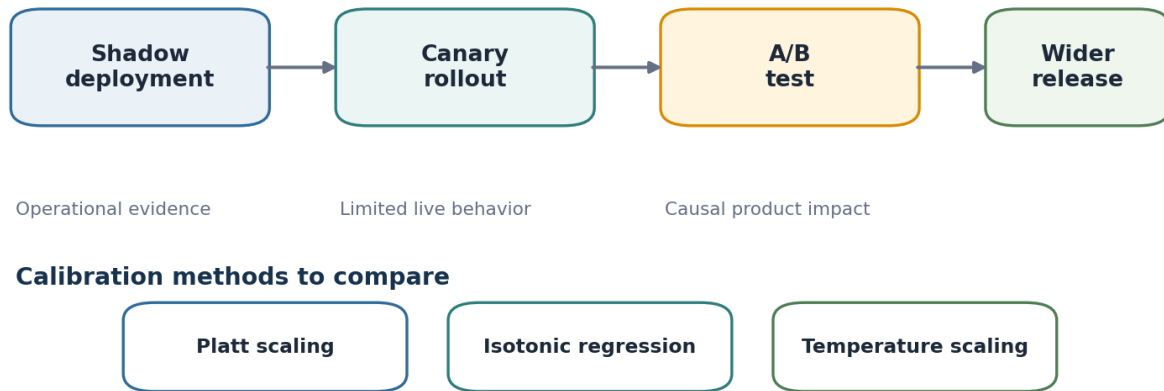


Figure 10: Validation and calibration

Prevent future-data leakage

Every feature and label must be computed relative to a prediction-time cutoff.

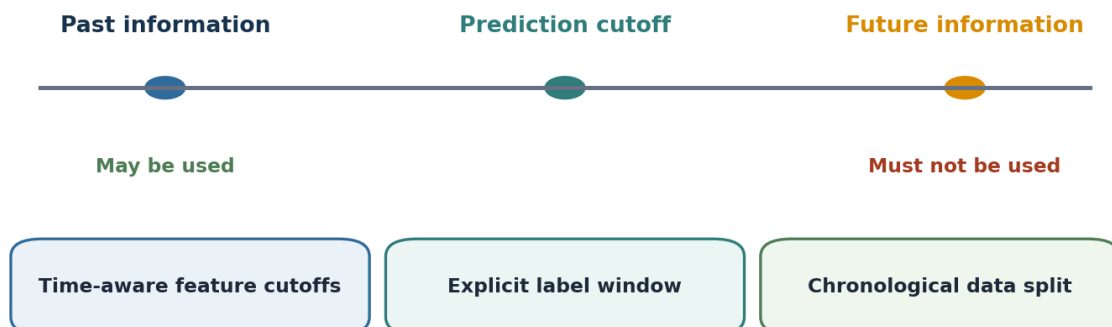


Figure 11: Leakage prevention

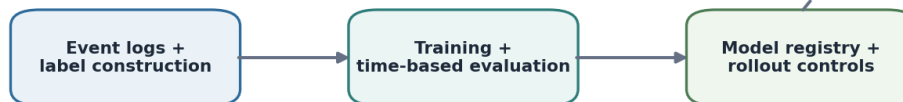
Reference architecture

One reasonable production-balanced design under the current assumptions.

ONLINE SERVING



OFFLINE LEARNING AND VALIDATION



Starting recommendation: two-tower or ANN retrieval for scale, a richer calibrated CTR model for final ranking, and explicit frequency and hide/block guardrails.

Figure 12: Reference architecture